Rapport de Projet – Developpement Web ING1

Plateforme Maison Intelligente (Spring Boot + React)

Khayem Ben Ghorbel Rindra Ziyad Anastasia Valerie

Annee universitaire 2025–2026

Table des matières

1 Introduction **2**

2 Objectifs et perimetre **2**
2.1 Objectifs fonctionnels . . . 2
2.2 Objectifs non-fonctionnels . . . 2

3 Architecture technique **2**
3.1 Stack . . . 2
3.2 Organisation du code . . . 2
3.3 Routes REST principales . . . 2

4 Methodologie projet **3**
4.1 Repartition des contributions . . . 3

5 Couverture fonctionnelle **3**
5.1 Module Information . . . 3
5.2 Module Visualisation . . . 3
5.3 Module Gestion . . . 3
5.4 Module Administration . . . 3

6 Securite et controle d'acces **4**

7 UML des cas d'utilisation (complet) **4**

8 Diagramme de sequence critique (inscription -> activation) **6**

9 Schema UML complet (grande vue) **7**

10 Schema de base de donnees complet (grande vue relationnelle) **9**
10.1 Choix de modelisation et performances . . . 11
10.2 Dictionnaire de donnees detaille (schema physique) . . . 11
10.3 Regles d'integrite et de gestion . . . 13

11 Tests et validation **13**
11.1 Builds . . . 13
11.2 Tests de parcours demo . . . 13
11.3 Responsive et UX . . . 13

12 Deploiement **13**

13 Difficultes rencontrees et solutions **13**

14 Conclusion **13**

1 Introduction

Ce document presente la realisation du projet de Developpement Web ING1 autour du theme **Maison Intelligente**. L'objectif est de livrer une application web complete, avec architecture full-stack, base de donnees persistante, gestion des roles, automatisation domotique et module d'administration.

Le projet respecte les attentes du module :

— frameworks backend/frontend,
— persistance des donnees,
— recherche multi-criteres,
— separation claire des modules,
— demonstration fonctionnelle complete.

2 Objectifs et perimetre

2.1 Objectifs fonctionnels

1. **Information** : consultation publique des pieces, objets et services.
2. **Visualisation** : espace membre (authentification, profil, consultation detaillee).
3. **Gestion** : pilotage avance des objets, scenarios, maintenance, statistiques.
4. **Administration** : gestion utilisateurs, droits et validation des comptes.

2.2 Objectifs non-fonctionnels

— interface responsive,
— securite d'acces par role et niveau,
— traces d'actions,
— disponibilite locale et deploiement cloud (Render).

3 Architecture technique

3.1 Stack

— **Backend** : Java 21, Spring Boot 3.3, Spring Data JPA, Validation, Mail SMTP.
— **Frontend** : React 19, Vite.
— **BDD** : H2 (fichier local) et base MySQL prete.
— **Deploiement** : Render (API Docker + Frontend statique).

3.2 Organisation du code

— `backend/` : modeles, DTO, repositories, services, controllers.
— `frontend/` : pages, composants, etat UI, appels REST.
— racine : `pom.xml`, `Dockerfile`, `render.yaml`, `README.md`.

3.3 Routes REST principales

— `/api/info/*`
— `/api/auth/*`
— `/api/visualisation/*`
— `/api/gestion/*`
— `/api/admin/*`

4 Methodologie projet

Le travail a ete mene de maniere iterative (inspire SCRUM) :

— priorisation du backlog et progression par increments,
— cycles courts implementation + verification,
— demonstrations regulieres des fonctions importantes,
— stabilisation finale pour soutenance.

4.1 Repartition des contributions

Membre	Role principal	Contributions
Khayem Ben Ghorbel	Pilotage technique	Architecture globale, integration front/back, securite auth, stabilisation demo
Rindra	Implementation fonctionnelle	Support modules Information/Visualisation, validations de parcours
Ziyad	Support backend	Verifications API, aides sur robustesse modele/controleurs
Anastasia	UX / presentation	Qualite visuelle des ecrans, coherence navigation et lisibilite
Valerie	Documentation / soutenance	Structuration rapport, support scenario de demonstration

5 Couverture fonctionnelle

5.1 Module Information

— affichage des pieces et objets,
— recherche avec filtres combines,
— acces sans connexion.

5.2 Module Visualisation

— inscription / connexion / profil,
— consultation des objets et des services,
— progression utilisateur (points + niveau).

5.3 Module Gestion

— CRUD objets connectes,
— commandes specialisees par type (machine cafe, aspirateur, etc.),
— scenarios domotiques,
— maintenance priorisee (CRITICAL/HIGH/MEDIUM),
— historique et statistiques.

5.4 Module Administration

— gestion des utilisateurs,
— attribution/retrait du role admin,
— validation/suspension des comptes,
— traitement des demandes de suppression.

6 Securite et controle d'acces

Le flux de securite applique est le suivant :

1. inscription,
2. verification email par code (envoi SMTP reel),
3. validation manuelle par administrateur,
4. connexion autorisee uniquement apres ces etapes.

Un compte non approuve est bloque au login (retour HTTP 403). Cette approche reduit les inscriptions sauvages et donne un controle operateur sur les nouveaux comptes.

Les acces aux modules sont verifies cote backend :

— module Gestion reserve aux utilisateurs de niveau `AVANCE`,
— module Administration reserve aux comptes `admin=true`.

7 UML des cas d'utilisation (complet)

Cette section decrit le comportement attendu du systeme en fonction des acteurs. Le diagramme couvre le visiteur, le membre, le membre avance et l'administrateur.

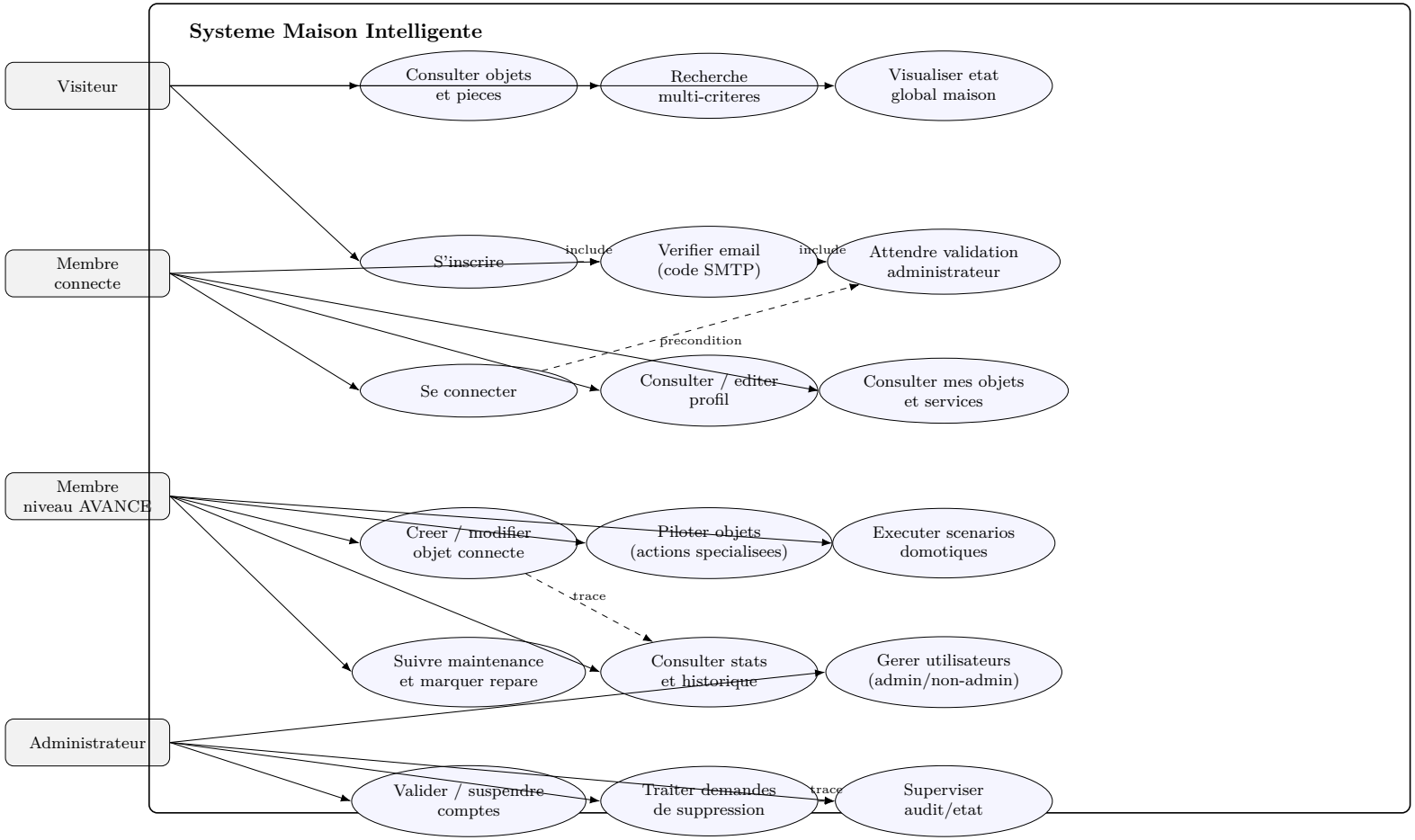

FIGURE 1 – Diagramme UML des cas d'utilisation (version complete).

8 Diagramme de sequence critique (inscription -> activation)

Le scenario ci-dessous detaille le flux securise applique pour tout nouveau compte.

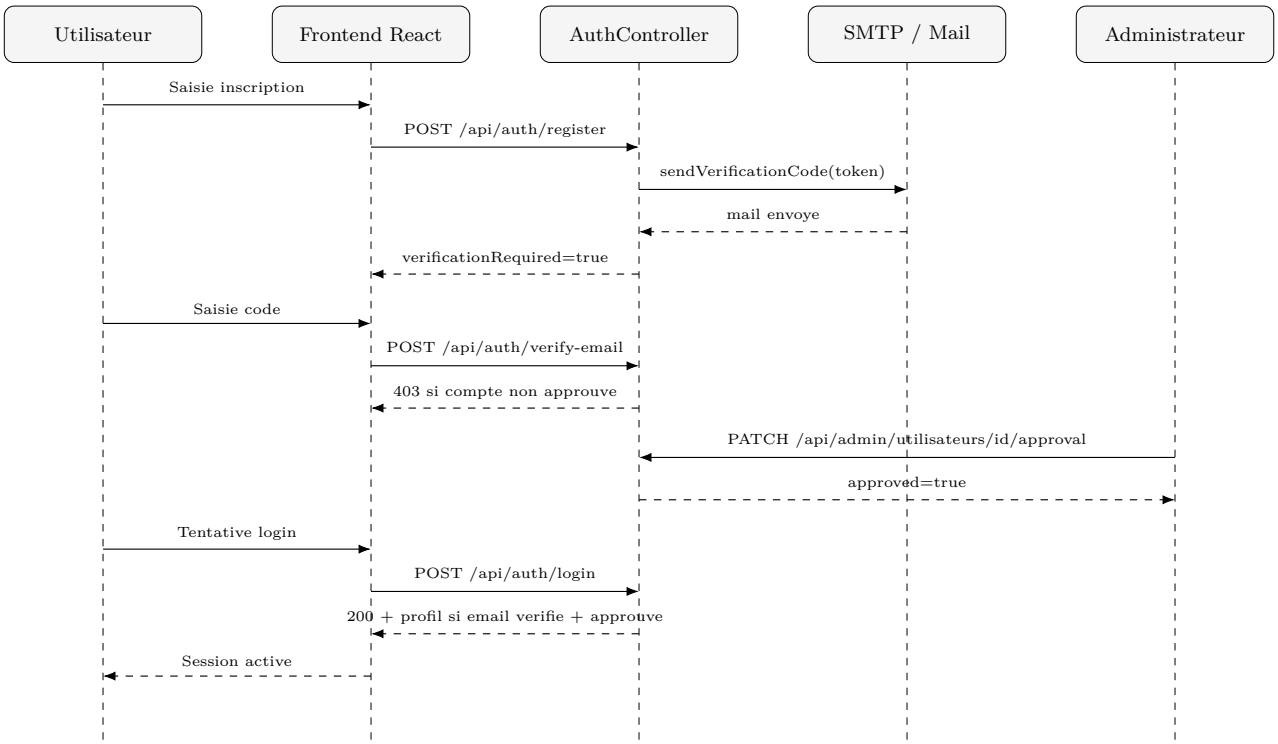

FIGURE 2 – Diagramme de sequence du parcours de securisation des comptes.

9 Schema UML complet (grande vue)

Le schema UML ci-dessous presente l'ensemble des entites metier, leurs heritages, leurs compositions et les liens majeurs entre modules.

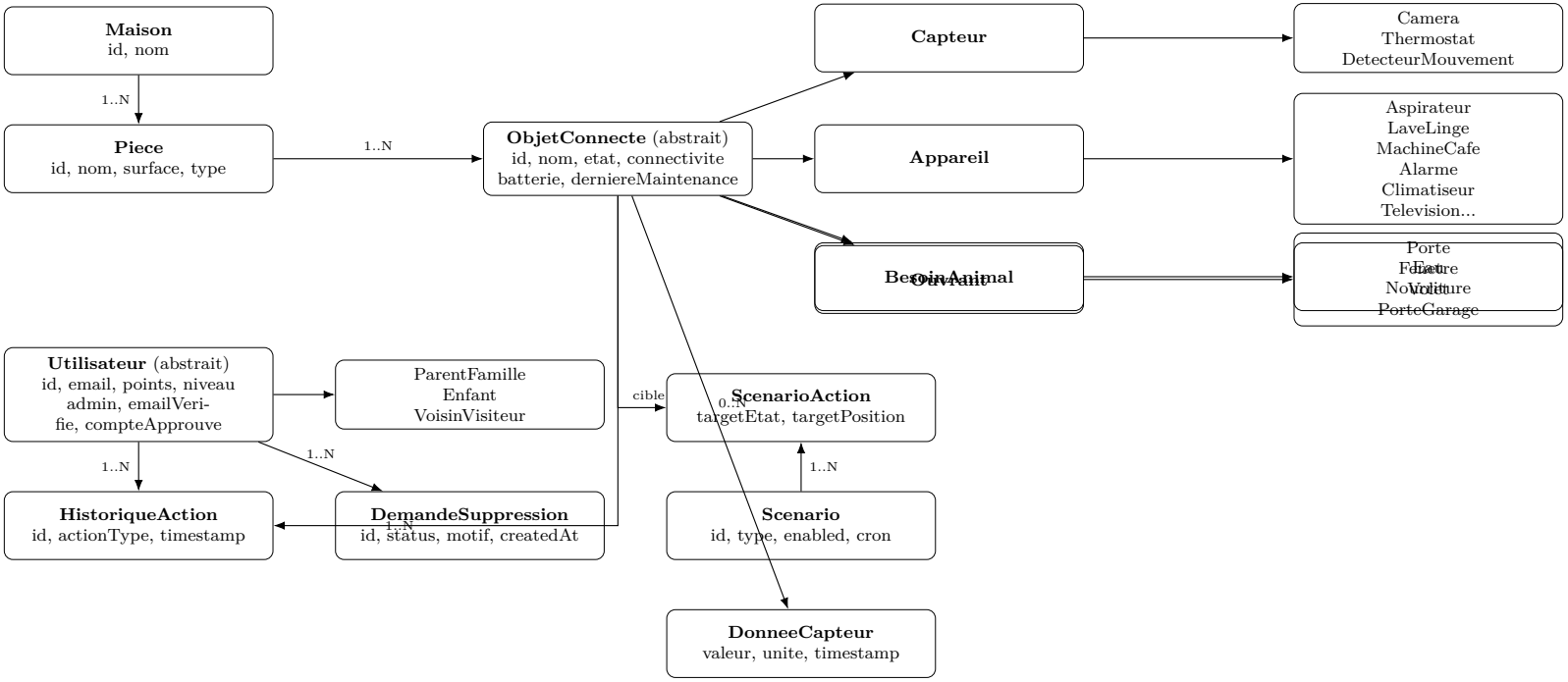

FIGURE 3 – Schema UML global (vue complete et agrandie).

10 Schema de base de donnees complet (grande vue relationnelle)

La base de donnees est organisee autour des tables coeur `utilisateur`, `objet_connecte`, `piece`, et des tables de liaison/fonctionnement (historique, scenarios, demandes, donnees capteurs).

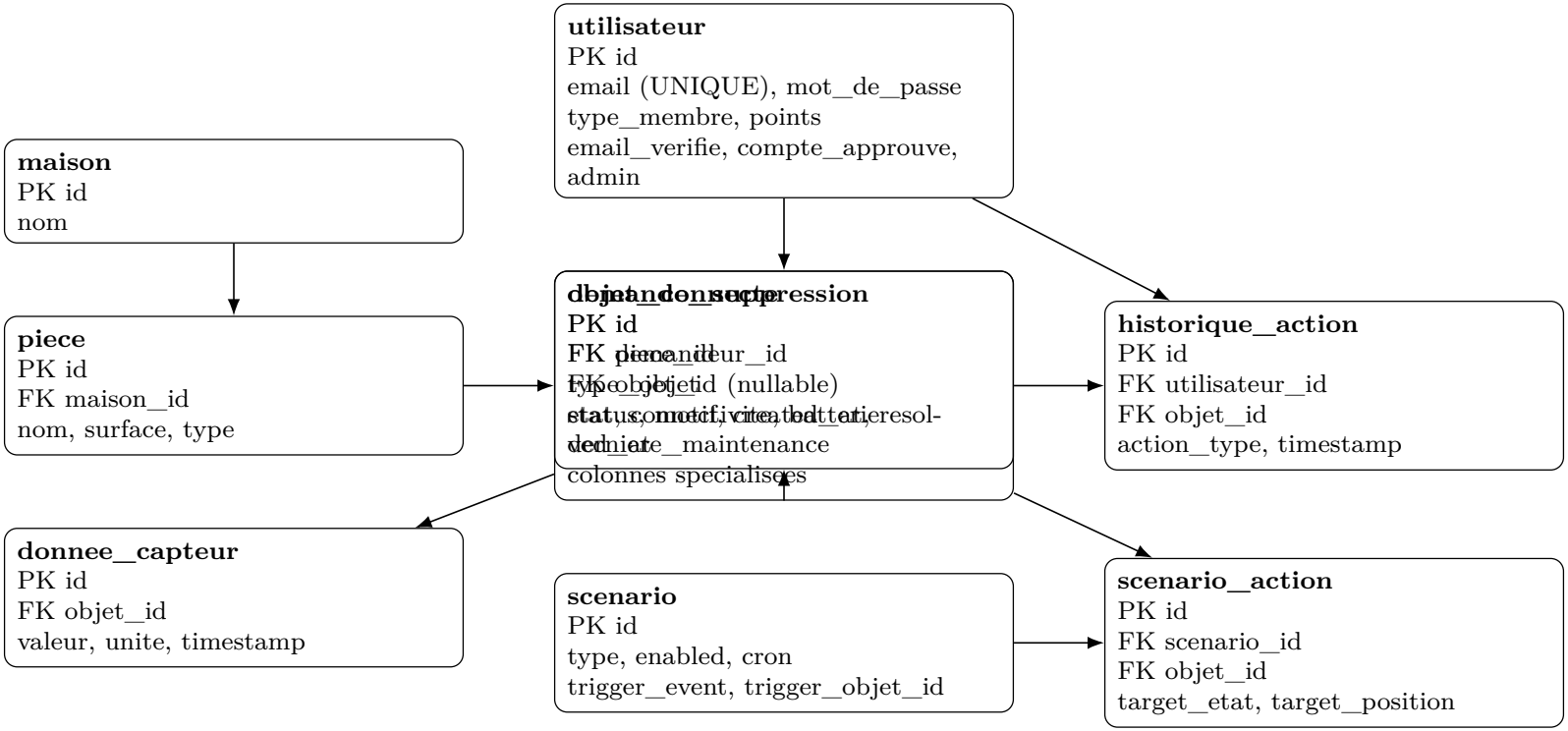

Figure 4 – Schema relationnel principal (PK/FK) de la base.

10.1 Choix de modelisation et performances

- Heritage JPA en `SINGLE_TABLE` pour `utilisateur` et `objet_connecte`.
- Integrite referentielle via FK sur les tables de traces et d'automatisation.
- Requetes cibles a indexer :
  - `objet_connecte(type_objet, piece_id, etat)`
  - `historique_action(utilisateur_id, objet_id, timestamp)`
  - `scenario(enabled, type, trigger_event)`

10.2 Dictionnaire de donnees detaille (schema physique)

Convention : PK = cle primaire, FK = cle etrangere, NN = not null, UQ = unique.

Table	Colonne	Type / contraintes	Description metier
maison	id	bigint, PK, NN	Identifiant maison
maison	nom	varchar, NN	Nom logique de la maison
piece	id	bigint, PK, NN	Identifiant piece
piece	maison_id	bigint, FK->maison.id, NN	Proprietaire maison
piece	nom	varchar, NN	Nom de la piece (salon, cuisine...)
piece	surface	float, nullable	Surface en m2
piece	type	varchar, nullable	Type de piece / specialisation
utilisateur	id	bigint, PK, NN	Identifiant utilisateur
utilisateur	type_membre	varchar, NN	Discriminator (PARENT/ENFANT/VOISIN)
utilisateur	email	varchar, UQ, NN	Login principal
utilisateur	mot_de_passe	varchar, NN	Hash du mot de passe
utilisateur	prenom, nom	varchar, NN	Identite de l'utilisateur
utilisateur	points	float, NN	Progression experience
utilisateur	admin	boolean, NN	Droit administration
utilisateur	email_verifie	boolean, NN	Email valide via token
utilisateur	compte_approuve	boolean, nullable	Validation manuelle admin
utilisateur	niveau derive	enum/applicatif	Niveau calculable depuis points
utilisateur	nb_connexions	integer, nullable	Compteur de connexions
utilisateur	date_inscription	timestamp, nullable	Date de creation compte
utilisateur	email_verification_token	varchar, nullable	Code OTP temporaire
utilisateur	email_verification_expire_tm	timestamp, nullable	Date d'expiration du code
objet_connecte	id	bigint, PK, NN	Identifiant objet
objet_connecte	type_objet	varchar, NN	Discriminator polymorphique
objet_connecte	piece_id	bigint, FK->piece.id	Piece d'appartenance
objet_connecte	nom	varchar, NN	Libelle objet
objet_connecte	etat	varchar, nullable	ACTIF/INACTIF/EN_PANNE...
objet_connecte	connectivite	varchar, nullable	WIFI, BLE, FILAIRE...
objet_connecte	batterie	float, nullable	Niveau batterie standard
objet_connecte	derniere_maintenance	timestamp, nullable	Date derniere intervention
objet_connecte	conso_energie	float, nullable	Consommation estimee
objet_connecte	position, zone, cycle	varchar, nullable	Parametres transverses appareils

Table	Colonne	Type / contraintes	Description metier
objet_connecte	niveau, niveau_reservoir	float, nullable	Reservoir/etat numerique
objet_connecte	animal	varchar, nullable	Cible besoins animaux
objet_connecte	portion_grammes	integer, nullable	Portion distribuee nourriture
objet_connecte	niveau_eau, niveau_cafe	float, nullable	Attributs MachineCafe
objet_connecte	total_preparations	integer, nullable	Stats MachineCafe
objet_connecte	derniere_boisson	varchar, nullable	Derniere boisson preparee
historique_action	id	bigint, PK, NN	Identifiant log
historique_action	utilisateur_id	bigint, FK->utilisateur.id	Auteur de l'action
historique_action	objet_id	bigint, FK->objet_connecte.id	Objet cible
historique_action	action_type	varchar, NN	Nature de l'action
historique_action	details	varchar/text, nullable	Detail libre
historique_action	timestamp	timestamp, NN	Horodatage
donnee_capteur	id	bigint, PK, NN	Identifiant mesure
donnee_capteur	objet_id	bigint, FK->objet_connecte.id	Capteur source
donnee_capteur	valeur	float, NN	Valeur mesuree
donnee_capteur	unite	varchar, nullable	Unite (C, %, dB...)
donnee_capteur	timestamp	timestamp, NN	Date mesure
scenario	id	bigint, PK, NN	Identifiant scenario
scenario	nom	varchar, NN	Nom metier scenario
scenario	type	varchar, NN	MANUAL / SCHEDULED / CONDITIONAL
scenario	enabled	boolean, NN	Activation scenario
scenario	cron_expr	varchar, nullable	Planification cron
scenario	trigger_event	varchar, nullable	Type evenement conditionnel
scenario	trigger_objet_id	bigint, FK nullable	Objet source evenement
scenario_action	id	bigint, PK, NN	Identifiant action scenario
scenario_action	scenario_id	bigint, FK->scenario.id, NN	Scenario parent
scenario_action	objet_id	bigint, FK->objet_connecte.id, NN	Objet cible
scenario_action	target_etat	varchar, nullable	Etat voulu
scenario_action	target_position	varchar, nullable	Position voulue
demande_suppression	id	bigint, PK, NN	Identifiant demande
demande_suppression	demandeur_id	bigint, FK->utilisateur.id, NN	Utilisateur initiateur
demande_suppression	objet_id	bigint, FK nullable	Objet cible suppression
demande_suppression	status	varchar, NN	PENDING/APPROVED/REJECTE
demande_suppression	motif	varchar/text, nullable	Justification utilisateur
demande_suppression	note_admin	varchar/text, nullable	Commentaire de traitement
demande_suppression	traite_par_id	bigint, FK nullable	Admin traitant
demande_suppression	created_at, resolved_at	timestamp	Debut / fin du workflow

10.3 Regles d'integrite et de gestion

— Un utilisateur ne peut pas se connecter si `email_verifie=false`.
— Un utilisateur verifie reste bloque si `compte_approuve=false`.
— Une action scenario doit viser un objet existant (FK non nulle).
— Les suppressions d'objets conservent la tracabilite admin.
— Les donnees capteurs sont historisees et jamais ecrasees en masse.

11 Tests et validation

11.1 Builds

— Backend : `mvn -DskipTests compile` -> succes.
— Frontend : `npm.cmd run build` -> succes.

11.2 Tests de parcours demo

1. verification endpoint `/api/health`,
2. inscription utilisateur,
3. verification email par code,
4. blocage login tant que validation admin absente,
5. validation admin puis connexion autorisee,
6. execution scenario et actions de maintenance.

11.3 Responsive et UX

— rendu teste en desktop/tablette/mobile,
— feedback utilisateur par toasts et etats explicites,
— sections sensibles masquee/verrouillees selon droits.

12 Deploiement

— API Render : service Docker, healthcheck `/api/health`
— Frontend Render : service statique Vite
— URL racine API (`/`) en 404 : comportement normal d'une API-only

13 Difficultes rencontrees et solutions

1. **Evolution du schema local H2** : gestion des colonnes legacy et compatibilite lors des evolutions modeles.
2. **Synchronisation etat front/back** : mise a jour locale immediate de l'UI apres reponse API pour eviter la latence percue.
3. **Securisation des inscriptions** : ajout verification email reelle + validation admin obligatoire avant login.

14 Conclusion

Le projet livre une plateforme complete, structuree et demonstrable, avec separation claire des modules, modelisation metier riche, automatisation domotique, et securite renforcee.

Les axes d'amelioration prioritaires pour une suite industrielle sont :

— migration full MySQL + scripts de migration formels,
— tests automatises E2E,
— observabilite (metriques, logs structures, alerting).

Annexe – Commandes utiles

```
# Backend
mvn spring-boot:run

# Frontend
cd frontend
npm.cmd run dev

# Build
mvn -DskipTests compile
cd frontend && npm.cmd run build
```